

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

Campus Santiago San Joaquín

Bases de Datos Avanzadas – Tarea 1: MongoDB

Segundo Semestre 2026

Estudiante: Alejandro Sánchez Bruna

Rol: 202273605-4

Repositorio GitHub: <https://github.com/4L3BKN/Tareas-BDDAvanzadas>

Para ver cualquiera de las imagenes ocupadas a lo largo del informe entrar a la carpeta *img/* del repositorio de github escrito al inicio. El JSON ocupado para poblar la base de datos también se encuentra en el github.

1. Declaración uso de IA

Para la creación del JSON ocupado a lo largo de la tarea se ocupó Gemini 3.5-Flashlite. La estructura base usada en este informe fue creada por el mismo modelo.

2. Set Up y Preparación de Datos

Para conectarse al replica set se tuvo que editar los host del computador para que reconociera los distintos nodos.

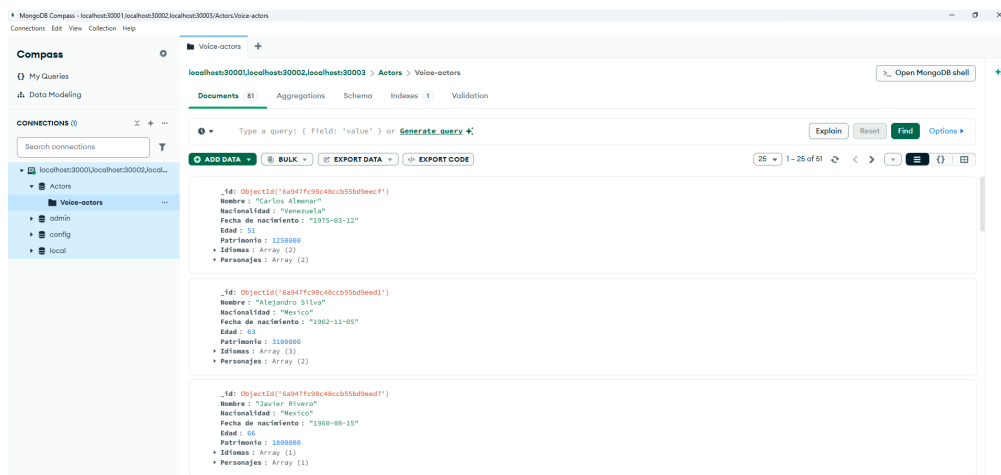


Figura 1: Conexión exitosa al Replica Set desde MongoDB Compass.

3. Parte 1: Consultas MQL

El uso de `.sort({Nombre : 1})`, `.limit(10)` y `{_id : 0, Nombre : 1, Nacionalidad : 1, Edad : 1, Idiomas : 1}` se usará en todas la Parte 1, por que de esta forma se cumple que los resultados

esten ordenados ascendentemente según el nombre, limitado a 10 resultados y que solo se muestren 4 campos incluyendo el Nombre.

3.1. Consulta 1: Proyección, filtrado y ordenamiento sobre campos simples

Descripción: Se muestra actores o actrices de voz mayores de 50 años.

```
1 db["Voice-actors"].find(  
2   {Edad:{$gt: 50}},  
3   {_id:0, Nombre:1, Nacionalidad:1, Edad: 1,Idiomas: 1},  
4 ).limit(10).sort({Nombre: 1})
```

Listing 1: Consulta MQL para mayores de 50 años

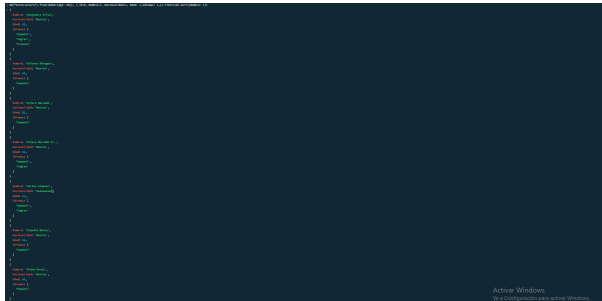


Figura 2: Primeros registros del resultado



Figura 3: Registros restantes del resultado.

3.2. Consulta 2: Búsqueda dentro de un arreglo simple

Descripción: Se buscó las actrices o los actores de nacionalidad chilena que también tengan el inglés entre sus idiomas. Tienen la misma configuración de orden que la anterior consulta.

```
1 db["Voice-actors"].find(  
2   {$and : [ {Nacionalidad: 'Chile'}, {Idiomas: 'Ingles'}]},  
3   {_id:0, Nombre: 1, Nacionalidad: 1, Edad: 1, Idiomas: 1}  
4 ).sort({Nombre: 1}).limit(10)
```

Listing 2: Consulta MQL para actrices o actores con nacionalidad chilena y que sepan ingles

```
{  
  "Nombre": "Valeria Zuniga",  
  "Nacionalidad": "Chile",  
  "Edad": 25,  
  "Idiomas": [  
    "Espanol",  
    "Ingles"  
  ]  
}
```

Figura 4: Resultado de la consulta 2 en MongoDB Compass.

3.3. Consulta 3: Búsqueda sobre arreglos de documentos embebidos

Descripción: Son los actores y actrices menores de 30 años que le den voz a un personaje principal en un genero de acción. En esta ocasión se proyectó personajes en vez de idiomas, para así demostrar que la consulta si estaba retornando lo correcto.

```

1 db["Voice-actors"].find(
2   {Edad: {$lt: 30},
3     Personajes: {
4       $elemMatch: {
5         Rol: 'principal',
6         Generos: 'Accion'
7       }
8     },
9   },
10  {_id:0, Nombre:1, Edad:1,Nacionalidad:1, Personajes: 1}
11 ).sort({Edad:1}).limit(10)

```

Listing 3: Consulta MQL menores de 30 años con personaje principal en el genero de accion

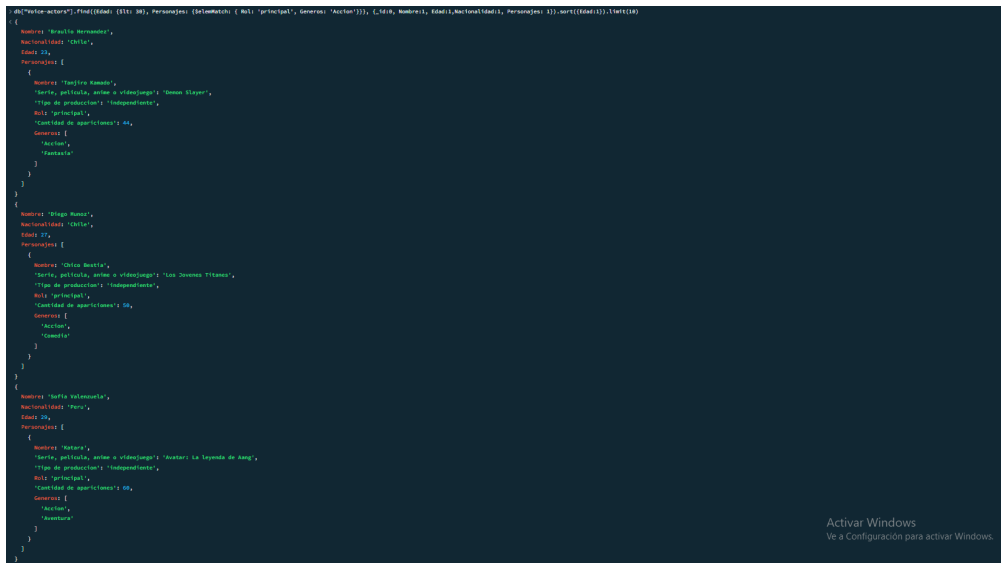


Figura 5: Resultado de la consulta 3 en MongoDB Compass.

3.4. Consulta 4: Pipeline de agregación

Descripción: Con aggregate se consiguió ordenar por patrimonio en orden descendente, luego se saltaron los 4 primeros documentos, limitaron 10 documentos. Muestra el Nombre, Edad, Patrimonio, Nacionalidad de cada actor/actriz.

```

1 db["Voice-actors"].aggregate([
2   {
3     $sort: {
4       Patrimonio: -1
5     }
6   },
7   {
8     $skip: 4
9   },
10  {
11    $limit:
12      10
13  },
14  {
15    $project:

```

```
16     {
17         _id: 0,
18         Nombre: 1,
19         Edad: 1,
20         Patrimonio: 1,
21         Nacionalidad: 1
22     }
23 }
24 ])
```

Listing 4: Consulta MQL aggregate



Figura 6: Primeros registros del resultado



Figura 7: Registros restantes del resultado.

4. Parte 2: Procesamiento de consultas e Índices

4.1. Antes de la creación de índice

1. Consulta 1 - OperationTime: 1788710329

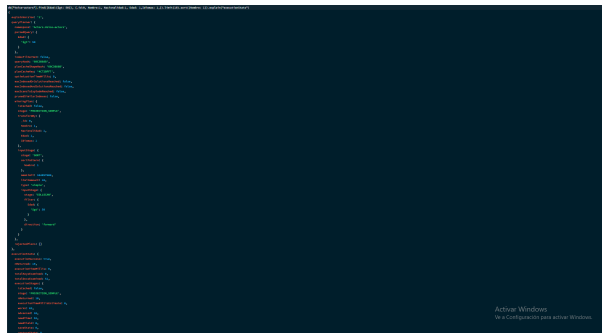


Figura 8: Primera parte del explain

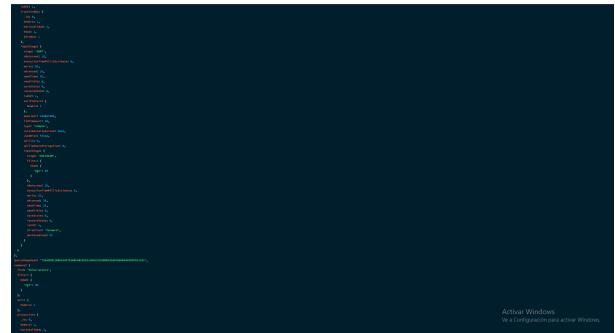


Figura 9: Segunda parte del explain

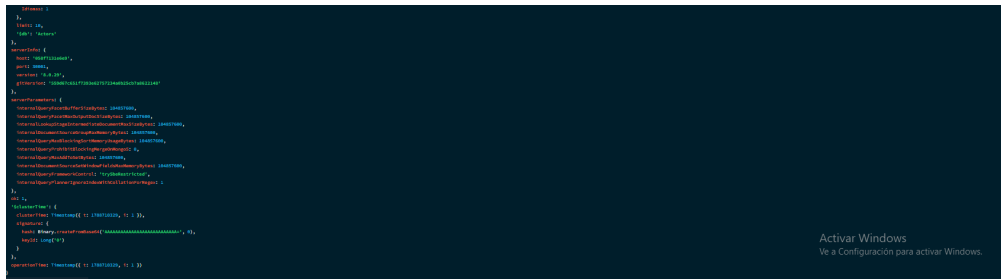


Figura 10: Tercera parte del explain para la Consulta 1

2. Consulta 2 - OperationTime: 1788710599

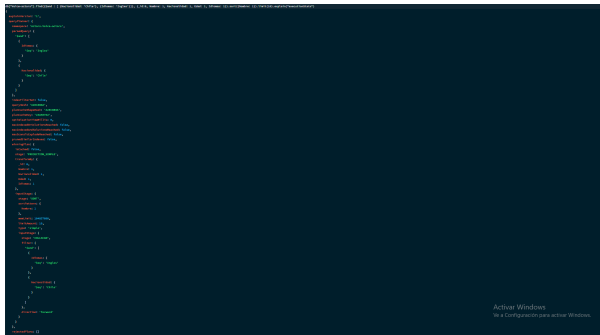


Figura 11: Primera parte del explain

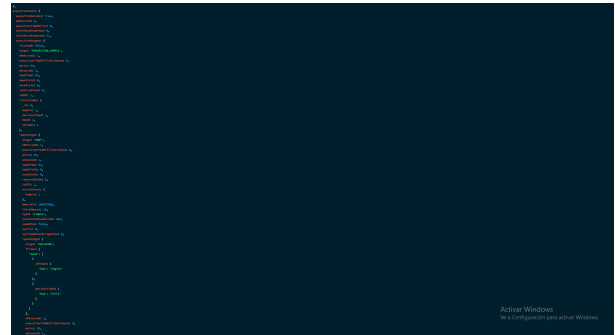


Figura 12: Segunda parte del explain

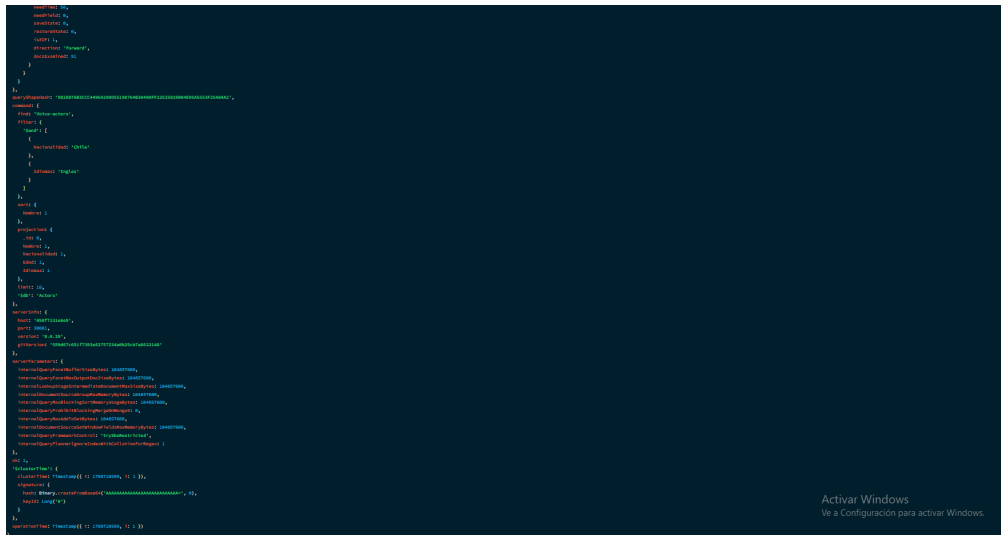


Figura 13: Tercera parte del explain

3. Consulta 3 - OperationTime: 1788710849

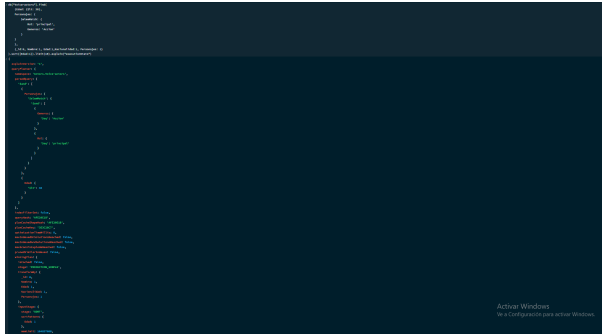


Figura 14: Primera parte del explain

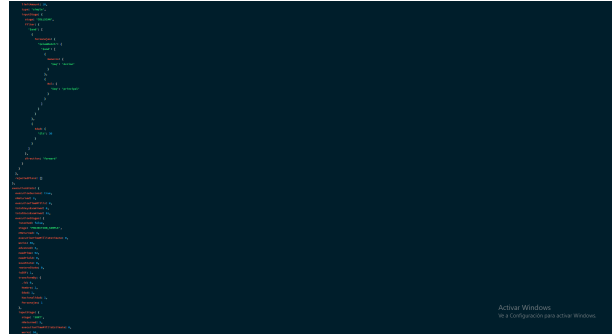


Figura 15: Segunda parte del explain

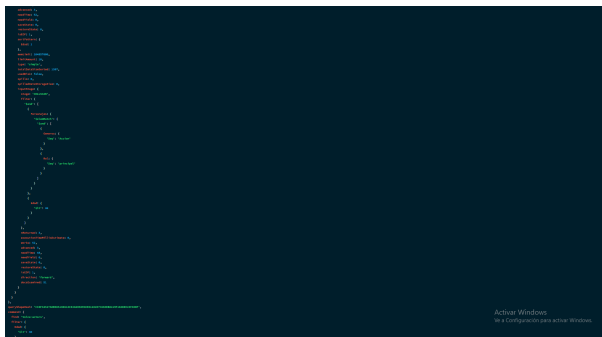


Figura 16: Tercera parte del explain

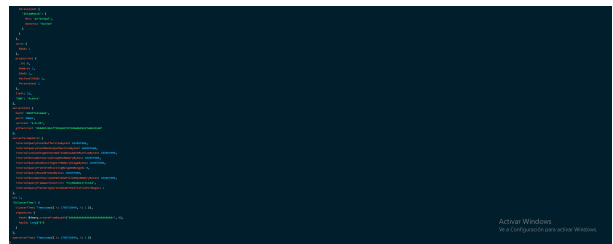


Figura 17: Cuarta parte del explain

4. Aggregate - OperationTime: 1788711299

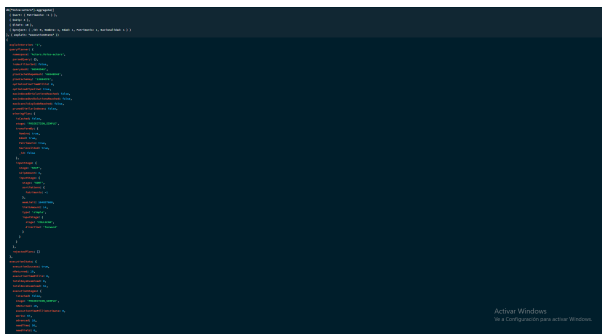


Figura 18: Primera parte del explain

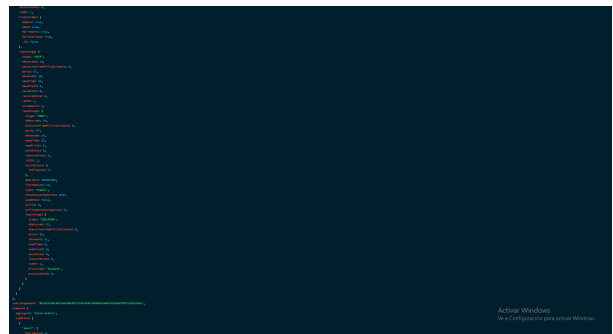


Figura 19: Segunda parte del explain

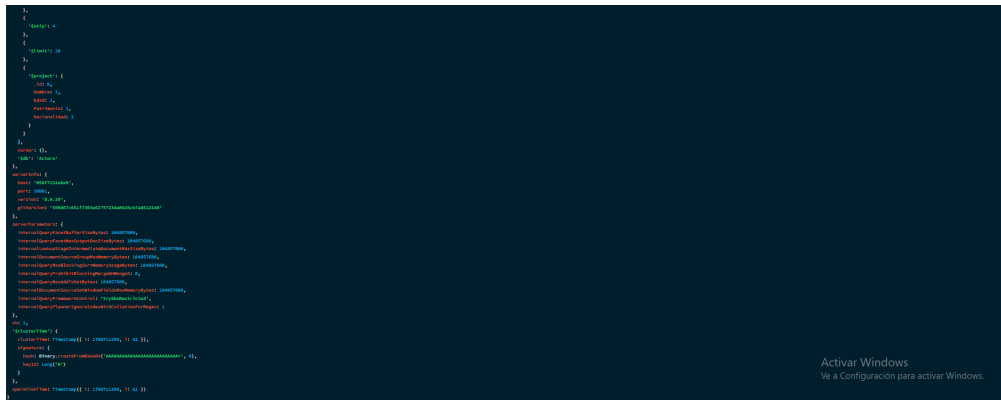


Figura 20: Tercera parte del explain

4.2. Después de la creación del índice

La creación de los índices fueron con las siguientes queries:

```
1 db["Voice-actors"].createIndex({ Nacionalidad: 1 })
```

Listing 5: Creación índice sobre Nacionalidad

```
1 db["Voice-actors"].createIndex({ Edad: 1 })
```

Listing 6: Creación índice sobre Edad

1. Consulta 1 - con indice Nacionalidad y Edad - OperationTime: 1788713139

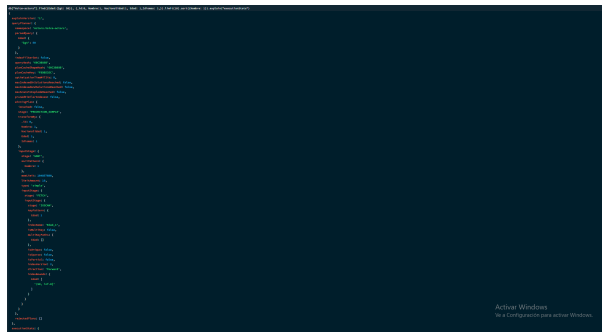


Figura 21: Primera parte del explain

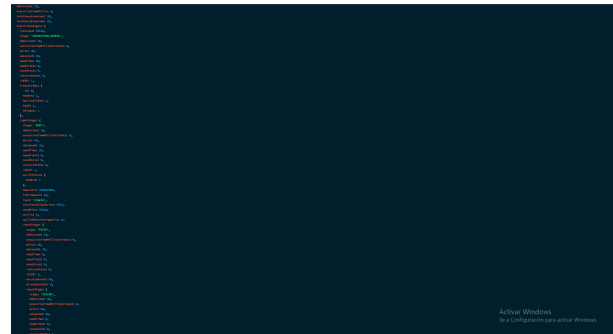


Figura 22: Segunda parte del explain

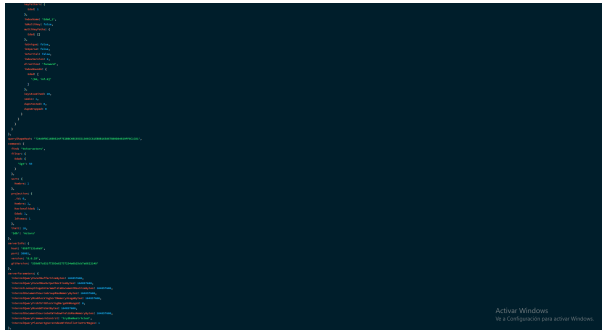


Figura 23: Primera parte del explain



Figura 24: Cuarta parte del explain

2. Consulta 2 - con indice Nacionalidad - OperationTime: 1788712469

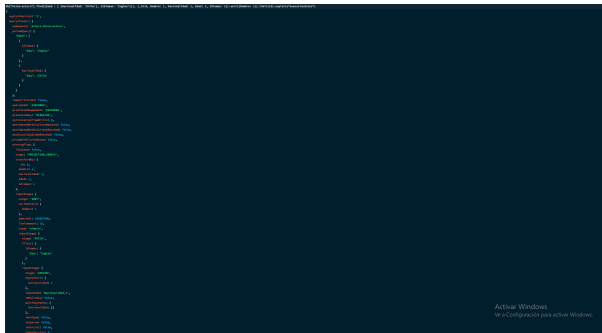


Figura 25: Primera parte del explain

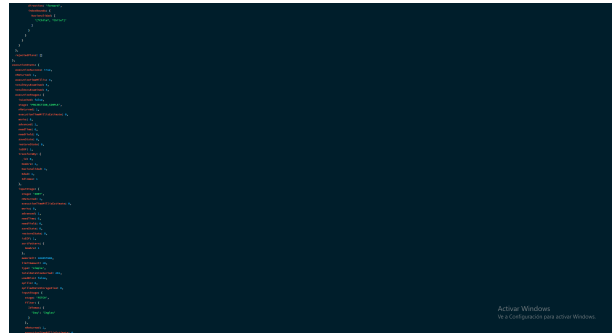


Figura 26: Segunda parte del explain

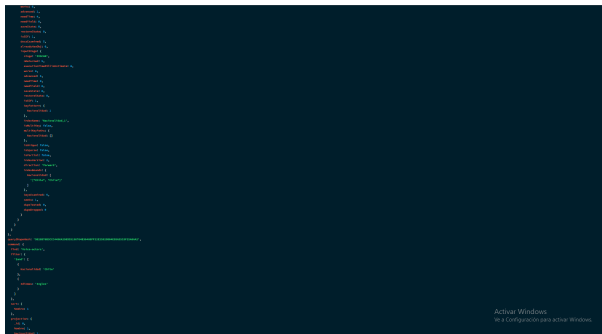


Figura 27: Tercera parte del explain

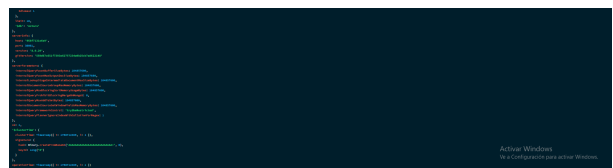


Figura 28: Segunda parte del explain

3. Consulta 3 - con índice Nacionalidad y Edad - OperationTime: 1788713639

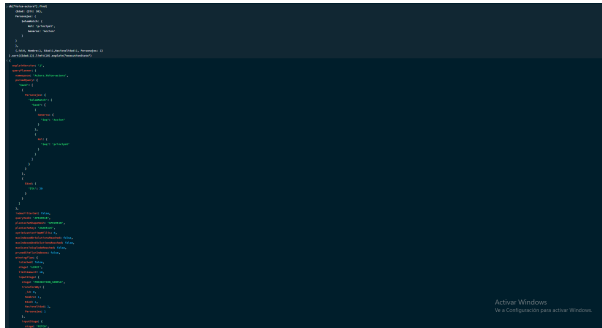


Figura 29: Primera parte del explain

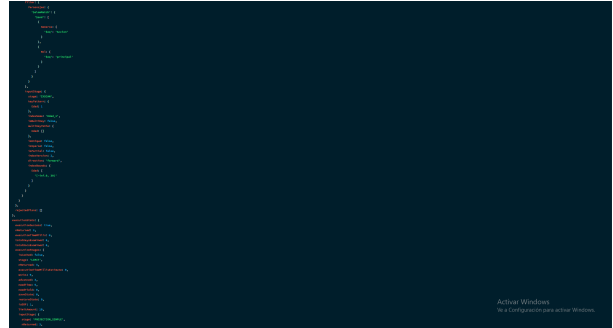


Figura 30: Segunda parte del explain

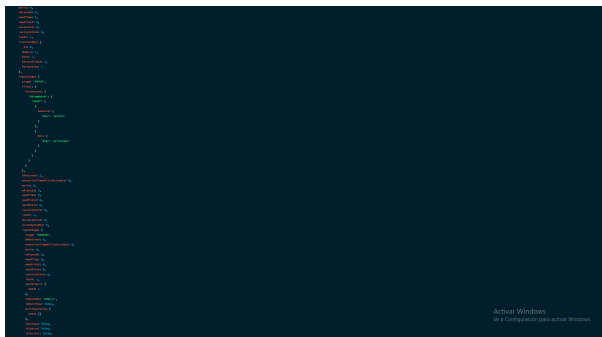


Figura 31: Tercera parte del explain

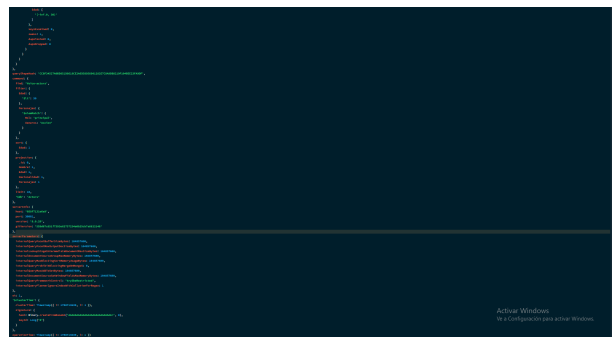


Figura 32: Cuarta parte del explain

4. Aggregate - con índice Nacionalidad y Edad - OperationTime: 1788714949

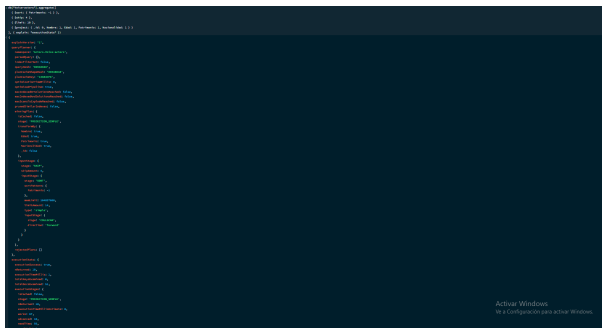


Figura 33: Primera parte del explain

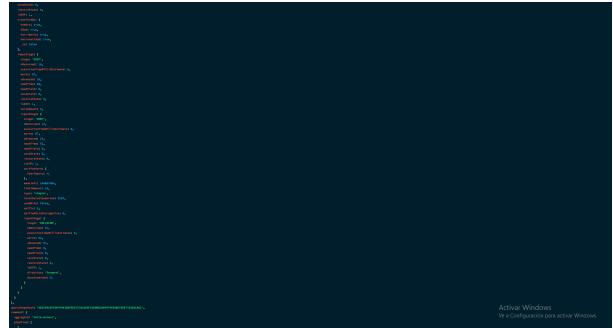


Figura 34: Segunda parte del explain

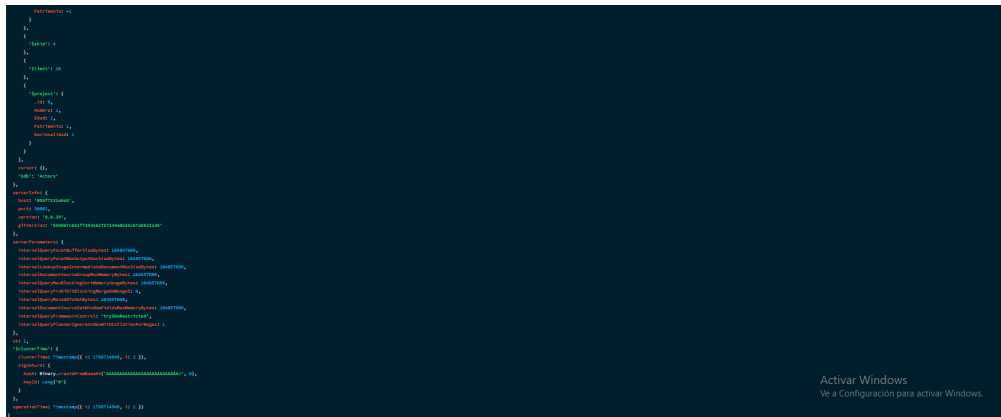


Figura 35: Tercera parte del explain

4.3. Comparaciones

4.3.1. Consulta 1 (Filtro por Edad mayores a 50)

A continuación se comparan las métricas de ejecución de la Consulta 1 antes y después de aplicar un índice sobre el campo Edad:

■ Sin Índice:

- **Estrategia:** COLLSCAN (escaneo secuencial de toda la colección).
- **Documentos examinados (totalDocsExamined):** 51 documentos. Esto es el 100 % de la base de datos.
- **Claves examinadas (totalKeysExamined):** 0.

■ Con Índice (Edad_1):

- **Estrategia:** IXSCAN seguido de una etapa FETCH.
- **Documentos examinados (totalDocsExamined):** Disminuyó a 28 documentos, solo aquellos que superan los 50 años.
- **Claves examinadas (totalKeysExamined):** 28 claves del índice.
- **Documentos retornados (nReturned):** 10, delimitados por el límite de la consulta.

Conclusión del análisis: El índice basado en rangos permite acceder de forma directa a los actores mayores de 50 años, evitando la lectura innecesaria de los documentos de actores más jóvenes que no cumplen con la condición del filtro.

4.3.2. Consulta 2 (Filtro por Nacionalidad e Idioma)

A continuación se comparan las métricas de ejecución de la Consulta 2 antes y después de aplicar un índice sobre el campo Nacionalidad:

■ Sin Índice:

- **Estrategia:** Utilizó COLLSCAN.
- **Documentos examinados (totalDocsExamined):** 51 documentos.

- **Claves examinadas (totalKeysExamined):** 0.
- **Con Índice (Nacionalidad_1):**
 - **Estrategia:** Utilizó IXSCAN apoyado por una etapa FETCH.
 - **Documentos examinados (totalDocsExamined):** Se redujo a solo 5 documentos.
 - **Claves examinadas (totalKeysExamined):** 5 claves del índice.
 - **Documentos retornados (nReturned):** 1, se mantiene consistente.

Conclusión del análisis: La creación del índice evita que el motor de base de datos realice un recorrido lineal por toda la colección, acotando la búsqueda de manera eficiente a los documentos que coinciden con el criterio de nacionalidad antes de aplicar el filtro secundario.

4.3.3. Consulta 3 (Filtro por Edad e Arreglo de Personajes)

A continuación se comparan las métricas de ejecución de la Consulta 3 antes y después de aplicar el índice sobre el campo Edad:

- **Sin Índice:**
 - **Estrategia:** COLLSCAN acompañado de una etapa de SORT.
 - **Documentos examinados (totalDocsExamined):** 51 documentos.
 - **Claves examinadas (totalKeysExamined):** 0.
- **Con Índice (Edad_1):**
 - **Estrategia:** IXSCAN combinado con un FETCH para el filtro anidado en el arreglo.
 - **Documentos examinados (totalDocsExamined):** Reducido a 8 documentos, actores menores de 30 años.
 - **Claves examinadas (totalKeysExamined):** 8 claves del índice.
 - **Documentos retornados (nReturned):** 3 resultados consistentes.

Conclusión del análisis: El índice optimiza de forma notable la consulta al filtrar de entrada por rango de edad, evitando la revisión completa de la colección y proveyendo los datos en el orden solicitado, lo que previene operaciones de ordenamiento costosas en memoria.

4.3.4. Consulta 4 (Pipeline de Agregación con Ordenamiento)

A continuación se analizan las métricas del pipeline de agregación que ordena de forma descendente por Patrimonio, omitiendo los primeros registros (\$skip) y limitando el resultado (\$limit):

- **Sin Índice y Con Índice Inadecuado (Nacionalidad-Edad):**
 - **Estrategia:** Ambas ejecuciones coinciden exactamente en su plan de ejecución, utilizando un escaneo completo de la colección (COLLSCAN) y una etapa de SORT.
 - **Documentos examinados (totalDocsExamined):** 51 documentos.
 - **Claves examinadas (totalKeysExamined):** 0, el índice existente fue ignorado por el optimizador al no coincidir con el criterio de ordenamiento.

Conclusión del análisis: Este comportamiento demuestra que los índices deben diseñarse específicamente en función de los campos involucrados en los filtros y ordenamientos de las consultas. Como el índice disponible no incluía el campo **Patrimonio**, el motor de MongoDB no pudo utilizarlo para evitar el escaneo completo ni el ordenamiento intensivo, evidenciando que una indexación incorrecta o desacoplada no aporta mejoras de rendimiento.

4.4. Análisis de Escalabilidad

Si la colección pasa de contener 51 documentos a **10.000.000.000 de documentos**, el comportamiento del sistema y la relevancia del rendimiento cambian drásticamente:

- **Consultas ejecutadas sin índices (COLLSCAN):** Se vuelven costosas. El motor de base de datos se ve obligado a realizar un recorrido secuencial de toda la colección ($O(N)$), leyendo gigabytes o terabytes de datos innecesariamente desde el disco e incurriendo en un cuello de botella masivo de entrada/salida.
- **Consultas que utilizan índices (IXSCAN):** Mantienen un rendimiento altamente eficiente y predecible. Gracias a las estructuras de árbol B-Tree, la complejidad de búsqueda se reduce a un orden logarítmico ($O(\log N)$), permitiendo ubicar las referencias exactas en cuestión de milisegundos sin importar el tamaño total de la base de datos.
- **Cantidad de documentos examinados (totalDocsExamined):** Sin índice, el indicador se dispararía a los 10 mil millones de documentos en cada consulta. Con un índice adecuado, el motor restringe drásticamente el universo de lectura, examinando únicamente los registros que cumplen estrictamente con el criterio de filtrado mediante la fase **FETCH**.
- **Tiempo y recursos necesarios:** Sin índices, el consumo de CPU y memoria RAM saturaría el servidor, provocando problemas de intercambio a disco (*thrashing*) debido a la imposibilidad de procesar ordenamientos masivos (**SORT**) en memoria. Con índices, el tiempo de respuesta se mantiene bajo y el uso de recursos es mínimo.
- **Almacenamiento adicional requerido por los índices:** Los índices consumen espacio físico en disco y memoria RAM (para mantener los árboles B-Tree en caché). A una escala de miles de millones de registros, un índice mal planificado o redundante puede acaparar gigabytes valiosos de almacenamiento, por lo que su diseño debe ser estratégico.

4.4.1. Conclusión sobre el diseño de índices a partir de explain()

Las pruebas realizadas a baja escala demuestran que el escaso volumen de datos en desarrollo enmascara los errores de diseño, haciendo que un **COLLSCAN** parezca imperceptible (*0ms*). Sin embargo, al proyectar las métricas observadas con **explain()** hacia un entorno de gran escala, se evidencia que:

1. El diseño de índices no es una simple optimización menor, sino un **requisito crítico de arquitectura**.
2. El parámetro **totalDocsExamined** deja de ser una estadística menor y pasa a representar la diferencia exacta entre una consulta que responde instantáneamente y una que satura y tumba un servidor en producción.

5. Parte 3: Flexibilidad, Resiliencia y Actualización

5.1. Flexibilidad de esquema

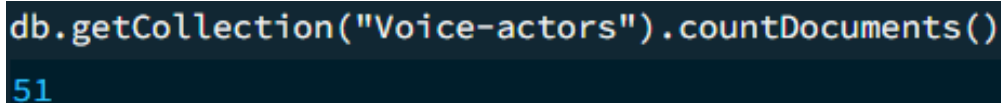
- **¿MongoDB permitiría insertar el documento sin el atributo “Fecha de nacimiento”?**
Sí se puede. MongoDB es una base de datos orientada a documentos con un esquema flexible. Por ende, no exige que todos los documentos de una misma colección posean exactamente los mismos campos.
- **¿Qué consecuencias podría tener posteriormente?**
Si se realizan queries basadas en el atributo faltante, dichos documentos serán ignorados o generarán inconsistencias a menos que se implemente un manejo explícito de valores ausentes mediante operadores como `$exists`.
- **¿Considera conveniente permitir documentos con atributos faltantes en este caso?**
Aunque la flexibilidad de esquemas es ideal para datos opcionales, los atributos en estos documentos, como la fecha de nacimiento o el nombre, deberían ser estrictamente obligatorios para garantizar la calidad de la información.
- **¿Existe alguna forma de controlar o validar la estructura de los documentos en MongoDB?**
Sí existe. MongoDB permite implementar validación estricta a nivel de esquema utilizando las reglas de **JSON Schema Validation** mediante el parámetro `validator` en el comando `createCollection` o `collMod`), permitiendo definir campos obligatorios (`required`), tipos de datos y restricciones adicionales.

5.2. Resiliencia de la base de datos

- **¿Funcionó la consulta? ¿Por qué?**
Sí funciona, ya que los nodos comparten copias de los documentos. Por ende, pueden responder sin el principal, solo deben elegir uno nuevo. Hice un

```
db.getCollection("Voice - actors").countDocuments()
```

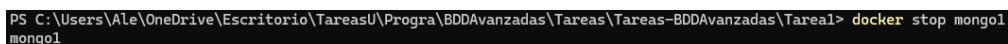
antes y después de botar el nodo principal.



```
db.getCollection("Voice-actors").countDocuments()  
51
```

Figura 36: consulta antes de botar el nodo principal

Se botó el mongo1 para comprobar el funcionamiento.



```
PS C:\Users\Ale\OneDrive\Escritorio\TareasU\Progra\BDDAvanzadas\Tareas\Tareas-BDDAvanzadas\Tarea1> docker stop mongo1  
mongo1
```

Figura 37: Comando usado para botar el nodo

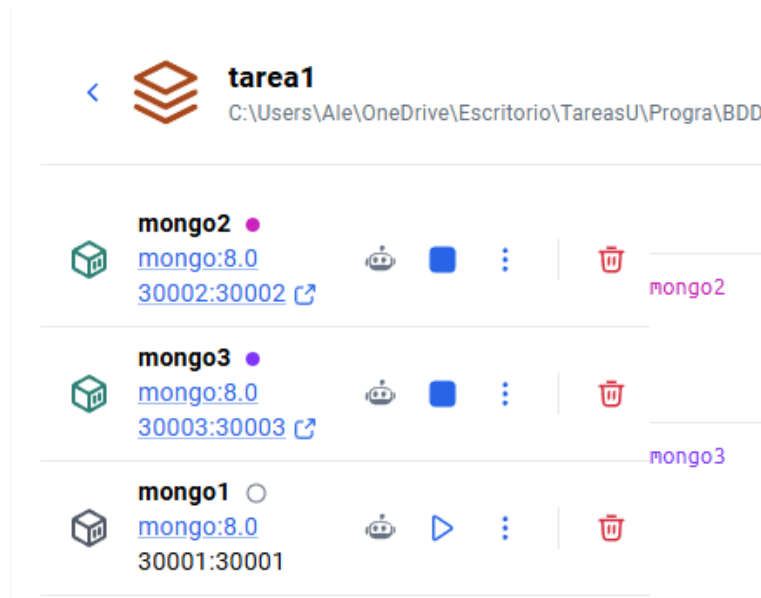


Figura 38: Evidencia docker del nodo caído

Una vez transcurrido el tiempo de reasignación del Replica Set, el clúster eligió un nuevo primario y la consulta se ejecutó exitosamente.

```
db.getCollection("Voice-actors").countDocuments()
51
```

Figura 39: consulta después de botar el nodo principal

5.3. Actualización masiva de documentos

Para actualizar sin realizar modificaciones documento por documento, se utilizó una operación de actualización masiva con pipeline de agregación condicional, aplicando los incrementos de patrimonio y la inserción del nuevo personaje de forma simultánea:

```
1 db.getCollection("Voice-actors").updateMany(
2   {},
3   [
4     {
5       $set: {
6         Patrimonio: { $round: [{ $multiply: ["$Patrimonio", 1.32] }, 2] },
7         Personajes: {
8           $concatArrays: [
9             "$Personajes",
10            [
11              {
12                Nombre: "Juan Ball Race",
13                "Serie, pelicula, anime o videojuego": "Sansa Ball Race",
14                "Tipo de produccion": "profesional",
15                Rol: {
16                  $cond: { if: { $lt: ["$Edad", 40] }, then: "principal",
else: "secundario" } }
```

```
17     },
18     "Cantidad de apariciones": {
19         $cond: { if: { $lt: ["$Edad", 40] }, then: 3, else: 1 }
20     },
21     Generos: ["Accion", "Fantasia"]
22 }
23 ]
24 ]
25 }
26 }
27 }
28 ]
29 );
```

Listing 7: Comando de actualización masiva en MongoDB

5.3.1. Evidencia: Antes y Después de la Actualización

Para ver los cambios se usó la query:

- Para los menores de 40:

```
1 db["Voice-actors"].find(
2     {Edad: {$lt: 40}},
3     {_id:0, Nombre:1, Patrimonio: 1, Personajes: 1}
4 ).sort({Edad:1}).limit(2)
5
```

Listing 8: Comando consulta menores de 40 años

- Para los mayores o iguales de 40:

```
1 db["Voice-actors"].find(
2     {Edad: {$gte: 40}},
3     {_id:0, Nombre:1, Patrimonio: 1, Personajes: 1}
4 ).sort({Edad:1}).limit(2)
5
```

Listing 9: Comando consulta 40+ años

1. Caso 1: Actores/Actrices menores de 40 años

Consulta antes de la actualización:

```

db["voice-actors"].find(
  (Edad: {$lt: 40}),
  [_id:0, Nombre:1, Patrimonio: 1, Personajes: 1]
).sort((Edad:1)).limit(2)
{
  Nombre: 'Braulio Hernandez',
  Patrimonio: 80000,
  Personajes: [
    {
      Nombre: 'Tanjiro Kamado',
      'Serie, película, anime o videojuego': 'Demon Slayer',
      'Tipo de producción': 'independiente',
      Rol: 'principal',
      'Cantidad de apariciones': 44,
      Generos: [
        'Acción',
        'Fantasía'
      ]
    }
  ]
}
{
  Nombre: 'Valeria Zuniga',
  Patrimonio: 120000,
  Personajes: [
    {
      Nombre: 'Alya Césaire',
      'Serie, película, anime o videojuego': 'Miraculous Ladybug',
      'Tipo de producción': 'independiente',
      Rol: 'secundario',
      'Cantidad de apariciones': 35,
      Generos: [
        'Animación',
        'Acción'
      ]
    }
  ]
}
}

```

Figura 40: consulta menores a 40 años antes de actualizar los documentos

Consulta después de la actualización:

```

db["voice-actors"].find(
  (Edad: {$lt: 40}),
  [_id:0, Nombre:1, Patrimonio: 1, Personajes: 1]
).sort((Edad:1)).limit(2)
{
  Nombre: 'Braulio Hernandez',
  Patrimonio: 10000,
  Personajes: [
    {
      Nombre: 'Tanjiro Kamado',
      'Serie, película, anime o videojuego': 'Demon Slayer',
      'Tipo de producción': 'independiente',
      Rol: 'principal',
      'Cantidad de apariciones': 44,
      Generos: [
        'Acción',
        'Fantasía'
      ]
    }
  ],
  {
    Nombre: 'Juan Ball Bana',
    'Serie, película, anime o videojuego': 'Sanzo Ball Bana',
    'Tipo de producción': 'profesional',
    Rol: 'principal',
    'Cantidad de apariciones': 3,
    Generos: [
      'Acción',
      'Fantasía'
    ]
  }
]
}
{
  Nombre: 'Valeria Zuniga',
  Patrimonio: 10000,
  Personajes: [
    {
      Nombre: 'Alya Césaire',
      'Serie, película, anime o videojuego': 'Miraculous Ladybug',
      'Tipo de producción': 'independiente',
      Rol: 'secundario',
      'Cantidad de apariciones': 35,
      Generos: [
        'Animación',
        'Acción'
      ]
    }
  ],
  {
    Nombre: 'Juan Ball Bana',
    'Serie, película, anime o videojuego': 'Sanzo Ball Bana',
    'Tipo de producción': 'profesional',
    Rol: 'principal',
    'Cantidad de apariciones': 3,
    Generos: [
      'Acción',
      'Fantasía'
    ]
  }
]
}

```

Figura 41: Consulta menores a 40 años después de actualizar los documentos

- **Antes:** Patrimonio de 80,000 y de 120,000. Sin el personaje de la nueva serie.

- **Después:** Patrimonio incrementado en 32% 105,600 y 158,400. Además la adición del personaje Juan Ball Race con rol "principal" y 3 apariciones.

2. Caso 2: Actores/Actrices de 40 años o más

Consulta antes de la actualización:

```
db["Voice-actors"].find(
  {Edad: {$gte: 40}},
  {_id:0, Nombre:1, Patrimonio: 1, Personajes: 1}
).sort({Edad:1}).limit(2)
{
  Nombre: 'Monica Estrada',
  Patrimonio: 700000,
  Personajes: [
    {
      Nombre: 'Tsunade',
      'Serie, película, anime o videojuego': 'Naruto Shippuden',
      'Tipo de producción': 'profesional',
      Rol: 'secundario',
      'Cantidad de apariciones': 110,
      Generos: [
        'Acción',
        'Anime'
      ]
    }
  ]
}
{
  Nombre: 'Wireya Mondoza',
  Patrimonio: 650000,
  Personajes: [
    {
      Nombre: 'Erza Scarlet',
      'Serie, película, anime o videojuego': 'Fairy Tail',
      'Tipo de producción': 'profesional',
      Rol: 'principal',
      'Cantidad de apariciones': 175,
      Generos: [
        'Fantasía',
        'Acción'
      ]
    }
  ]
}
```

Figura 42: consulta 40+ años antes de actualizar los documentos

Consulta después de la actualización:

```

db["voice-actors"].find(
  {State: ["Spain", "MEX"]},
  {_id: 1, Nombre: 1, Patrimonio: 1, Personajes: 1}
).sort([["id"], [-1]]).pretty()
{
  "Nombre": "Mónica Estrada",
  "Patrimonio": 924000,
  "Personajes": [
    {
      "Nombre": "Faustina",
      "Serie, película, anime o videojuego": "Haruto Shipyard",
      "Tipo de producción": "profesional",
      "Rol": "secundario",
      "Cantidad de apariciones": 10,
      "Generos": [
        "Mystery",
        "Action"
      ]
    },
    {
      "Nombre": "Juan Ball Race",
      "Serie, película, anime o videojuego": "Santo Ball Race",
      "Tipo de producción": "profesional",
      "Rol": "secundario",
      "Cantidad de apariciones": 1,
      "Generos": [
        "Mystery",
        "Fantasy"
      ]
    }
  ],
  "Nombre": "Mirage Mendoza",
  "Patrimonio": 858000,
  "Personajes": [
    {
      "Nombre": "Eras Scarlet",
      "Serie, película, anime o videojuego": "Fairy Tail",
      "Tipo de producción": "profesional",
      "Rol": "primario",
      "Cantidad de apariciones": 17,
      "Generos": [
        "Fantasy",
        "Action"
      ]
    },
    {
      "Nombre": "Juan Ball Race",
      "Serie, película, anime o videojuego": "Santo Ball Race",
      "Tipo de producción": "profesional",
      "Rol": "secundario",
      "Cantidad de apariciones": 1,
      "Generos": [
        "Mystery",
        "Fantasy"
      ]
    }
  ]
}

```

Figura 43: consulta 40+ años después de actualizar los documentos

- **Antes:** Patrimonio de 700,000 y 650,000. Sin el personaje de la nueva serie.
- **Después:** Patrimonio incrementado en 32% 924,000 y 858,000. Además la adición del personaje Juan Ball Race con rol "secundario" y 1 aparición.

6. Parte 4: Cierre

Para eliminar la base de datos se usó (`db.dropDatabase()`). El mensaje del termino fue el siguiente:

```

> db.dropDatabase()
< { ok: 1, dropped: 'Actors' }

```

Figura 44: cierre base de datos